RAPPORT DE PROJET · CSC 242 · TP 3 · 2025-2026

Simulation de SolarWinds Network Performance Monitor en programmation orientée objet

Essowe Kenneth Aloua · IPNET Institute of Technology, Lomé

Cours	CSC 242 · Algorithmique et programmation orientée objet
Filière	Réseaux, Systèmes, Sécurité et Cybersécurité
Livrables	Tableau des classes, algorithmes, programme Python
Programme	Un fichier Python d'environ 830 lignes, 20 classes, bibliothèque standard uniquement
Code source	github.com/EssoweKennethAloua4-sky/simulation-solarwinds-npm

En une phrase : plutôt que d'installer un logiciel de supervision payant, j'en ai reproduit le fonctionnement en Python : le programme découvre un petit réseau, interroge chaque appareil, enregistre les mesures et lève une alerte quand un seuil est dépassé.

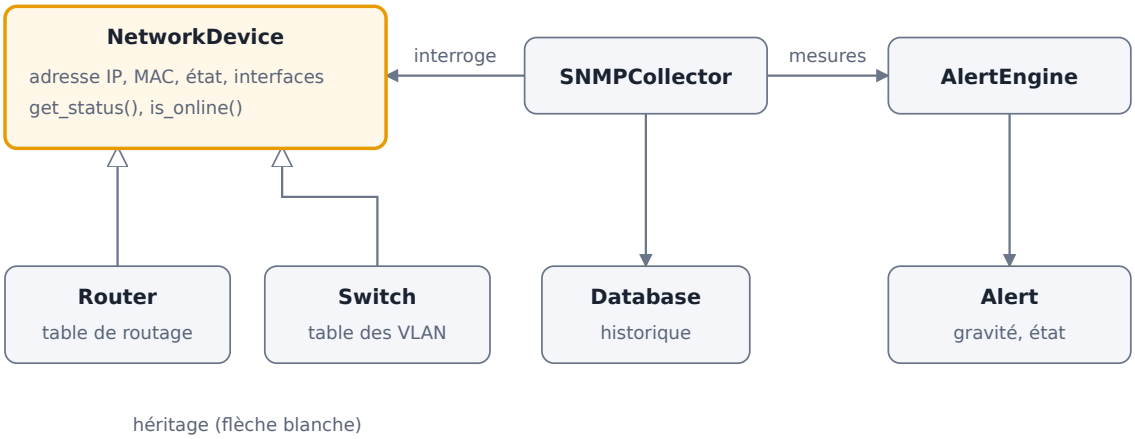
1 Le sujet

SolarWinds NPM (Network Performance Monitor) est un logiciel commercial utilisé en entreprise pour surveiller les routeurs, les commutateurs et les serveurs. Il interroge les équipements avec SNMP, un protocole standard : l'outil pose une question (« quelle est ta charge processeur ? ») et l'équipement répond.

Le TP demandait de modéliser ce système en programmation orientée objet, puis de l'implémenter. La programmation orientée objet consiste à découper un programme en « objets » qui ressemblent aux choses réelles : un routeur, un commutateur, une alerte. Chaque objet a des données (ses attributs) et des actions (ses méthodes).

2 La modélisation

J'ai d'abord rédigé un tableau des classes : pour chacune, ses attributs avec leur type et ses méthodes. Le point central est l'héritage : un routeur et un commutateur sont deux sortes d'appareils réseau. Ils reprennent tout ce que possède un appareil (adresse IP, adresse MAC, état, interfaces) et ajoutent ce qui leur est propre.



Les classes principales. Router et Switch héritent de NetworkDevice ; le collecteur interroge les appareils, transmet les mesures au moteur d'alertes et à la base de données.

3 Le programme

Le programme compte 20 classes réparties en sept blocs :

Bloc	Classes	Rôle
États	DeviceStatus, DeviceType, AlertSeverity…	Énumérations : les valeurs possibles d'un état (en ligne, dégradé, hors ligne…)
Réseau	NetworkDevice, Router, Switch, NetworkInterface, Metric	Les appareils, leurs interfaces et les mesures
SNMP	SNMPAgent, SNMPCollector	Simuler les questions-réponses SNMP et collecter les mesures
Découverte	NetworkDiscovery, NetworkTopologyMap	Trouver les appareils et dessiner la carte du réseau
Alertes	AlertRule, Alert, AlertEngine	Comparer chaque mesure à un seuil et créer les alertes
Stockage	Database	Garder l'historique des mesures et des alertes
Pilotage	Dashboard, NPMSystem	Afficher le tableau de bord et orchestrer les cycles

Les règles d'alerte

Mesure	Seuil	Gravité
Utilisation du processeur	> 80 %	Avertissement
Utilisation de la mémoire	> 85 %	Critique
Utilisation de la bande passante	> 90 %	Avertissement

4 Exécution

Au lancement, le système découvre quatre appareils (un routeur, un commutateur, un serveur de production et un serveur de sauvegarde), puis enchaîne cinq cycles : collecte, stockage, évaluation des alertes, affichage. Extrait d'une exécution réelle, au deuxième cycle :

```
CYCLE 2/5
[19:49:02] Collecte SNMP... ✓ (14 métriques)
[19:49:02] Stockage en BD... ✓ (30 total)
[19:49:02] Évaluation alertes... ✓ (1 actives)

=== TOPOLOGIE RÉSEAU ===
  - router-main (192.168.1.1) [ONLINE]
  - switch-floor1 (192.168.1.2) [ONLINE]
  - server-prod (192.168.1.10) [DEGRADED]
  - server-backup (192.168.1.11) [ONLINE]

[ALERTES] Actives: 1
  ⚠ [WARNING] [BANDWIDTH_USAGE] Seuil dépassé: 91.61 %
```

Sur l'ensemble de cette exécution : 5 cycles, 72 mesures collectées et 2 alertes levées. Les valeurs sont tirées au hasard à chaque lancement, donc chaque exécution est différente. En fin de simulation, le programme exporte un rapport au format JSON.

Pour le lancer : `python simulation_npm.py` (Python 3, aucune installation supplémentaire).

5 Ce que j'en retiens

- **Modéliser avant de coder :** le tableau des classes a servi de plan ; le code a suivi sans réorganisation majeure.
- **Héritage et énumérations :** éviter de répéter le même code pour chaque type d'appareil, et rendre les états lisibles.
- **Supervision vue de l'intérieur :** collecte, stockage, seuils et alertes sont les mêmes briques que dans Zabbix ou dans un SOC.
- **Limite :** les mesures sont simulées. L'étape suivante serait d'interroger de vrais équipements avec une bibliothèque SNMP.